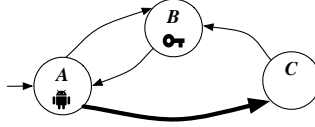


Part 1 - Artificial Intelligence (Time to complete Part 1: 100 minutes)

Consider the following scenario. A robot, *self*, can move across a network of locations shown in the picture below.



To traverse some edges of the network *self* needs a key (specifically the edge in **bold** in the picture).

Non-fluents predicates: $Location(x)$ denoting locations, in our case A, B, C ; $Obj(x)$ denoting the objects, in our case only the key *key*; $connected(x, y)$ where x and y are locations, denoting the edges not requiring a key in the network (see picture above); $connectedKey(x, y)$ where x and y are locations, denoting the edges that need the key to be traversed (see the picture above).

Fluents: $SelfAt(x)$ where x is a location, denoting that *self* is at location x ; $At(x, y)$ where x is an object and y a location, denoting that the object x is at location y ; $SelfHas(x)$ where x is an object, and denotes that *self* has object x with himself/herself.

Actions as available to *self*:

- $goto(\ell_1, \ell_2)$ to move from location ℓ_1 to location ℓ_2 . To do so *self* needs to be in location ℓ_1 , moreover if traversing the edge in the network needs the key, *self* need to have the key. The effect of $goto(\ell_1, \ell_2)$ is that *the new location of self*, together with all the objects that *self* has with him/her (i.e., the key), will be ℓ_2 .
- $pickup(obj)$ to pick up object *obj* (i.e., the key). To do so *self* needs to be in the same location as *obj*. The effect of $pickup(obj)$ is that *self* will have *obj* with himself/herself.
- ~~$drop(obj)$ to drop *obj* (i.e., the key). To do so *self* needs to have *obj* with himself/herself. The effect of $drop(obj)$ is that *self* will not have *obj* with himself/herself and the location of *obj* will be that of *self*.~~

Initially: *self* is at location A , the key is at location B , and *self* does not have any object with himself/herself. (The non-fluent predicates have their extension as above.)

Exercise 1. Formalize the above scenario as a Basic Action Theory in Reiter's Situation Calculus. Then check by regression:

1. Whether the action $goto(A, C)$ is executable in S_0 ;
2. Whether the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ is executable in S_0 ;
3. Whether, after executing the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ in S_0 , *self* has the key with himself/herself.

Exercise 2. Considering as goal $SelfAt(C)$, formalize the above scenario as a PDDL domain file and PDDL problem file. Then:

1. Draw the corresponding transition system;
2. Solve planning for achieving the above goal by using backward fixpoint computation, reporting the various stages of the fixpoint computation;
3. Solve planning for achieving the above goal by using forward depth-first search (uniformed), reporting the various steps of the forward search computation.

Exercise 3. Check, by using tableaux, whether the following FOL formula is indeed valid, and if not show a counterexample:
 $(\forall x.(A(x) \supset B(x))) \equiv ((\forall y.A(y)) \supset (\forall z.B(z)))$.

Exercise 4. Describe the DPLL procedure for satisfiability and use it to find a model (a satisfying interpretation) of the following set of clauses: $\{ \{A, B, \neg C\}, \{\neg A, B, C\}, \{\neg A\}, \{\neg B, C\} \}$.

1 Exercise 1: solution

Preliminary discussion. Let's write more rigorously the preconditions and effects described above.

- $goto(\ell_1, \ell_2)$
 - PRE: $SelfAt(\ell_1) \wedge (connected(\ell_1, \ell_2) \vee connectedKey(\ell_1, \ell_2) \wedge SelfHas(key))$
 - EFF: $(SelfAt(\ell_2) \wedge \neg SelfAt(\ell_1)) \wedge \forall x. PRE[SelfHas(x)] \supset At(x, \ell_2) \wedge \neg At(x, \ell_1)$
- $pickup(obj)$
 - PRE: $\exists \ell. SelfAt(\ell) \wedge At(obj, \ell)$
 - EFF: $SelfHas(obj)$

Basic Action Theory in Reiter's Situation Calculus. Now we formalize the scenario as a Basic Action Theory in Reiter's Situation Calculus.

- **Precondition Axioms.** They can be obtained immediately after the preliminary consideration above:

$$\begin{aligned} Poss(goto(\ell_1, \ell_2), s) &\equiv SelfAt(\ell_1, s) \wedge (connected(\ell_1, \ell_2) \vee connectedKey(\ell_1, \ell_2) \wedge SelfHas(key, s)) \\ Poss(pickup(obj), s) &\equiv \exists \ell. SelfAt(\ell, s) \wedge At(obj, \ell, s) \end{aligned}$$

- **Successor State Axioms.** To obtain them, we first write the effects using situations:

$$\begin{aligned} a = goto(\ell_1, \ell_2) &\supset \\ &(((SelfAt(\ell_2, do(a, s)) \wedge \neg SelfAt(\ell_1, do(a, s))) \wedge \\ &\forall x. (SelfHas(x, s) \supset At(x, \ell_2, do(a, s)) \wedge \neg At(x, \ell_1, do(a, s))))) \\ a = pickup(obj) &\supset SelfHas(obj, do(a, s)) \end{aligned}$$

Then we put them in normal form with only one fluent on the right-hand side of the implication. We can apply the following equivalence $[\forall \vec{x}. (\varphi_1(\vec{x}) \supset (\forall \vec{y}. \varphi_2(\vec{x}, \vec{y}) \supset \varphi_3(\vec{x}, \vec{y})))] \equiv [\forall \vec{x}, \vec{y}. (\varphi_1(\vec{x}) \wedge \varphi_2(\vec{x}, \vec{y}) \supset \varphi_3(\vec{x}, \vec{y}))]$:

$$\begin{aligned} a = goto(\ell_1, \ell_2) &\supset SelfAt(\ell_2, do(a, s)) \\ a = goto(\ell_1, \ell_2) &\supset \neg SelfAt(\ell_1, do(a, s)) \\ a = goto(\ell_1, \ell_2) \wedge SelfHas(x, s) &\supset At(x, \ell_2, do(a, s)) \\ a = goto(\ell_1, \ell_2) \wedge SelfHas(x, s) &\supset \neg At(x, \ell_1, do(a, s)) \\ a = pickup(obj) &\supset SelfHas(obj, do(a, s)) \end{aligned}$$

Note that all free variables in the axioms above are universally quantified. But if a variable appears only on the left-hand side of the implication we can apply the following equivalence $[\forall \vec{x}, \vec{y}. \varphi_1(\vec{x}, \vec{y}) \supset \varphi_2(\vec{x})] \equiv [\forall \vec{x}. (\exists \vec{y}. \varphi_1(\vec{x}, \vec{y}) \supset \varphi_2(\vec{x}))]$. Hence we get:

$$\begin{aligned} (\exists \ell_1. a = goto(\ell_1, \ell_2)) &\supset SelfAt(\ell_2, do(a, s)) \\ (\exists \ell_2. a = goto(\ell_1, \ell_2)) &\supset \neg SelfAt(\ell_1, do(a, s)) \\ (\exists \ell_1. a = goto(\ell_1, \ell_2)) \wedge SelfHas(x, s) &\supset At(x, \ell_2, do(a, s)) \\ (\exists \ell_2. a = goto(\ell_1, \ell_2) \wedge SelfHas(x, s)) &\supset \neg At(x, \ell_1, do(a, s)) \\ a = pickup(obj) &\supset SelfHas(obj, do(a, s)) \end{aligned}$$

Now by applying “*explanation closure*” we finally get the **successor state axioms**:

$$\begin{aligned} SelfAt(\ell, do(a, s)) &\equiv \exists \ell_1. a = goto(\ell_1, \ell) \vee SelfAt(\ell, s) \wedge \neg \exists \ell_2. a = goto(\ell, \ell_2) \\ At(x, \ell, do(a, s)) &\equiv \exists \ell_1. a = goto(\ell_1, \ell) \wedge SelfHas(x, s) \vee At(x, \ell, s) \wedge \neg (\exists \ell_2. a = goto(\ell, \ell_2) \wedge SelfHas(x, s)) \\ SelfHas(obj, do(a, s)) &\equiv a = pickup(obj) \vee SelfHas(obj, s) \end{aligned}$$

- **Initial situation description.** Remember all free variables universally quantified:

$$\begin{aligned} connected(\ell_1, \ell_2) &\equiv (\ell_1 = A \wedge \ell_2 = B) \vee (\ell_1 = B \wedge \ell_2 = A) \vee (\ell_1 = C \wedge \ell_2 = B) \text{ --it is not a fluent} \\ connectedKey(\ell_1, \ell_2) &\equiv (\ell_1 = A \wedge \ell_2 = C) \text{ --it is not a fluent} \\ SelfAt(\ell, S_0) &\equiv (\ell = A) \\ At(key, \ell, S_0) &\equiv (\ell = B) \\ \neg \exists x. SelfHas(x, S_0) & \end{aligned}$$

1. Check by regression whether the action $goto(A, C)$ is executable in S_0

$$\mathcal{R}[Poss(goto(A, C), S_0)] = SelfAt(A, S_0) \wedge (connected(A, C) \vee connectedKey(A, C) \wedge SelfHas(key, S_0))$$

Considering the initial situation description, we get that $connected(A, C)$ is false and while $connectedKey(A, C)$ is true, it is not the case that $SelfHas(key, S_0)$. So $goto(A, C)$ is not executable in S_0 .

2. Check by regression whether the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ is executable in S_0

We have to check each of the following:

$$\begin{aligned} Poss(goto(A, B), S_0) \\ Poss(pickup(key), S_1) \quad S_1 &= do(goto(A, B), S_0) \\ Poss(goto(B, A), S_2) \quad S_2 &= do(pickup(key), S_1) = do(pickup(key), do(goto(A, B), S_0)) \\ Poss(goto(A, C), S_3) \quad S_3 &= do(goto(B, A), S_2) = \\ &do(goto(B, A), do(pickup(key), S_1)) = do(goto(B, A), do(pickup(key), do(goto(A, B), S_0))) \end{aligned}$$

We check each of them by regression:

$$\mathcal{R}[Poss(goto(A, B), S_0)] = SelfAt(A, S_0) \wedge (connected(A, B) \vee connectedKey(A, B) \wedge SelfHas(key, S_0))$$

Considering the initial situation description this is true since $SelfAt(A, S_0)$ and $connected(A, B)$ both hold in the initial situation.

$$\begin{aligned} \mathcal{R}[Poss(pickup(key), S_1)] &= \exists \ell. \mathcal{R}[SelfAt(\ell, S_1)] \wedge \mathcal{R}[At(key, \ell, S_1)] \\ &= \exists \ell. \mathcal{R}[\exists \ell_1. goto(A, B) = goto(\ell_1, \ell) \vee \\ &\quad SelfAt(\ell, s) \wedge \neg \exists \ell_2. goto(A, B) = goto(\ell, \ell_2)] \wedge \mathcal{R}[At(key, \ell, do(goto(A, B), S_0))] \\ &= true \wedge \mathcal{R}[At(key, B, do(goto(A, B), S_0))] \\ \mathcal{R}[At(key, B, do(goto(A, B), S_0))] &= \exists \ell_1. goto(A, B) = goto(\ell_1, \ell) \wedge SelfHas(key, S_0) \vee \\ &\quad At(key, \ell, S_0) \wedge \neg (\exists \ell_2. goto(A, B) = goto(\ell, \ell_2) \wedge SelfHas(key, S_0)) \\ &= goto(A, B) = goto(A, B) \wedge SelfHas(key, S_0) \vee \\ &\quad At(key, B, S_0) \wedge \neg (\exists \ell_2. goto(A, B) = goto(B, \ell_2) \wedge SelfHas(key, S_0)) \\ &= SelfHas(key, S_0) \vee At(key, B, S_0) \\ &= true \end{aligned}$$

So $pickup(key)$ is indeed executable after $goto(A, B)$.

$$\begin{aligned} \mathcal{R}[Poss(goto(B, A), S_2)] &= \mathcal{R}[SelfAt(B, S_2)] \wedge (connected(B, A) \vee connectedKey(B, A) \wedge \mathcal{R}[SelfHas(key, S_2)]) \\ &= \mathcal{R}[SelfAt(B, S_2)] \wedge true \\ \mathcal{R}[SelfAt(B, do(pickup(key), S_1))] &= \exists \ell_1. pickup(key) = goto(\ell_1, B) \vee \mathcal{R}[SelfAt(B, S_1)] \wedge \neg \exists \ell_2. pickup(key) = goto(B, \ell_2) \\ &= false \vee \mathcal{R}[SelfAt(B, S_1)] \\ \mathcal{R}[SelfAt(B, do(goto(A, B), S_0))] &= \exists \ell_1. goto(A, B) = goto(\ell_1, B) \vee SelfAt(B, S_0) \wedge \neg \exists \ell_2. goto(A, B) = goto(B, \ell_2) \\ &= true \end{aligned}$$

So $goto(B, A)$ is indeed executable after $goto(A, B)$ followed by $pickup(key)$.

$$\begin{aligned} \mathcal{R}[Poss(goto(A, C), S_3)] &= \mathcal{R}[SelfAt(A, S_3)] \wedge (connected(A, C) \vee connectedKey(A, C) \wedge \mathcal{R}[SelfHas(key, S_3)]) \\ &= (true \wedge (false \vee true \wedge \mathcal{R}[SelfHas(key, S_3)])) \\ \mathcal{R}[SelfHas(key, do(goto(B, A), S_2))] &= goto(B, A) = pickup(key) \vee \mathcal{R}[SelfHas(key, S_2)] \\ &= false \vee \mathcal{R}[SelfHas(key, S_2)] \\ \mathcal{R}[SelfHas(key, do(pickup(key), S_1))] &= pickup(key) = pickup(key) \vee \mathcal{R}[SelfHas(key, S_1)] \\ &= true \end{aligned}$$

So $goto(A, C)$ is indeed executable after $goto(A, B)$ followed by $pickup(key)$, followed by $goto(B, A)$. So we can conclude that the sequence $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ is executable in S_0 .

3. Check by regression whether, after executing the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ in S_0 , *self* has the key with himself/herself. As before we use $S_1 = do(goto(A, B), S_0)$, $S_2 = do(pickup(key), S_1)$, $S_3 = do(goto(B, A), S_2)$ and $S_4 = do(goto(A, C), S_3)$.

$$\begin{aligned} \mathcal{R}[SelfHas(key, do(goto(A, C), S_3))] &= goto(A, C) = pickup(key) \vee \mathcal{R}[SelfHas(key, S_3)] \\ &= false \vee \mathcal{R}[SelfHas(key, S_3)] \\ \mathcal{R}[SelfHas(key, do(goto(B, A), S_2))] &= goto(B, A) = pickup(key) \vee \mathcal{R}[SelfHas(key, S_2)] \\ &= false \vee \mathcal{R}[SelfHas(key, S_2)] \\ \mathcal{R}[SelfHas(key, do(pickup(key), S_1))] &= pickup(key) = pickup(key) \vee \mathcal{R}[SelfHas(key, S_1)] \\ &= true \end{aligned}$$

Hence the projection above is indeed correct.

2 Exercise 2: solution

PDDL

- **Domain file:**

```
(define (domain ex2dom)
  (:requirements :adl)

  (:types
    object
    loc
  )

  (:constants
    key - object
  )

  (:predicates
    (connected ?l1 ?l2 - loc)      ;; nonfluent
    (connectedKey ?l1 ?l2 - loc)   ;; nonfluent
    (SelfAt ?l - loc)              ;; fluent
    (At ?o - object ?l - loc)      ;; fluent
    (SelfHas ?o - object)           ;; fluent
  )

  (:action goto
  :parameters (?l1 ?l2 - loc)
  :precondition
    (and (SelfAt ?l1) (or (connected ?l1 ?l2)
                          (and (connectedKey ?l1 ?l2) (SelfHas key))))
  :effect
    (and (SelfAt ?l2)
          (not (SelfAt ?l1))
          (forall (?o - object)
            (when (SelfHas ?o) (and (At ?o ?l2)
                                    (not (At ?o ?l1)))
          )
        )
    )
  )

  (:action pickup
  :parameters (?o - object)
  :precondition (exists (?l - loc) (and (SelfAt ?l) (At ?o ?l)))
  :effect (SelfHas ?o)
  )
)
```

- **Problem file:**

```
(define (problem ex2prob)
  (:domain ex2dom)
  (:objects
    key - object
    A - loc
    B - loc
    C - loc
  )
)
```

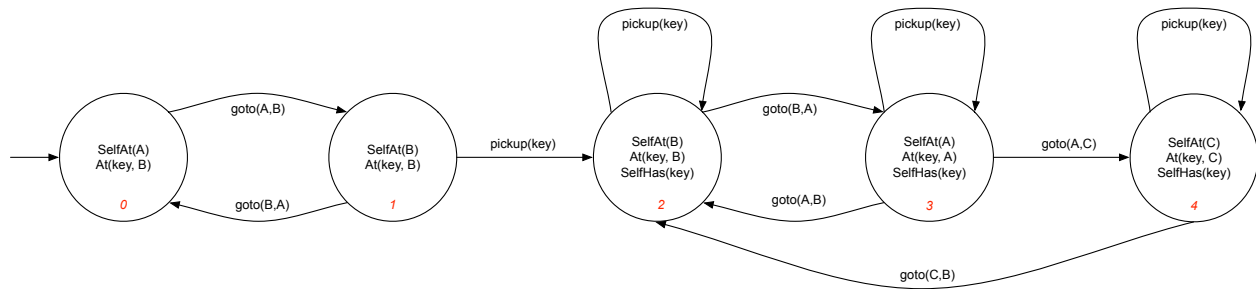
```

(:init
  (connected A B)
  (connected B A)
  (connectedKey A C)
  (connected C B)
  (SelfAt A)
  (At key B)
  ;; SelfHas is false for every object
)

(:goal
  (SelfAt C)
)

```

Corresponding transition system. Here is the transition system corresponding to the planning domain + initial state, considering that the only object present is the *key* (note every fluent atom not reported in the diagram is false):



Planning by backward fixpoint. We compute the plan by backward fixpoint:

$$\begin{aligned}
 Win_0 &= [[G]] &= \{4\} \\
 Win_1 &= Win_0 \cup PreE(Win_0) &= \{3, 4\} \\
 Win_2 &= Win_1 \cup PreE(Win_1) &= \{2, 3, 4\} \\
 Win_3 &= Win_2 \cup PreE(Win_2) &= \{1, 2, 3, 4\} \\
 Win_4 &= Win_3 \cup PreE(Win_3) &= \{0, 1, 2, 3, 4\} \\
 Win_5 &= Win_4 \cup PreE(Win_4) &= \{0, 1, 2, 3, 4\} \text{ fixpoint found!}
 \end{aligned}$$

Since the initial state 0 is in the fixpoint $Win = \bigcup_i Win_i$ we know that a plan exists, and we can actually compute a (universal) plan as follows:

$$\begin{aligned}
 \omega(0) &= goto(A, B) &-- & 0 \in Win_4, \delta(0, goto(A, B)) \in Win_3 \\
 \omega(1) &= pickup(key) &-- & 1 \in Win_3, \delta(1, pickup(key)) \in Win_2 \\
 \omega(2) &= goto(B, A) &-- & 2 \in Win_2, \delta(2, goto(B, A)) \in Win_1 \\
 \omega(3) &= goto(A, C) &-- & 3 \in Win_1, \delta(3, goto(A, C)) \in Win_0 \\
 \omega(4) &= win! &-- & \text{self is in goal state}
 \end{aligned}$$

Planning by deep-first search. Here is the algorithm:

```

plan DepthFirstSearch () {
  Stack t;
  t.push( (s0,empty) );
  while (!t.empty()) {
    StatePlan (s,p) = t.pop();
    if (!marked(s)) {
      mark(s);
      if (goal(s)) return p;
      forall (a in alpha(s))
        t.push( (delta(s,a), p+a) );
    }
  }
  return noplan;
}

```

We execute the algorithm on our problem (we intelligently perform the pushes in the stack, for simplicity).

```
t.push(0,empty)

(0,empty) <- t.pop()
mark(0)
t.push((1,goto(A,B)))

(1,goto(A,B)) <- t.pop()
mark(1)
t.push((0,goto(A,B);goto(B,A)))
t.push((2,goto(A,B);pickup(key)))

(2,goto(A,B);pickup(key)) <- t.pop()
mark(2)
t.push((2,goto(A,B);pickup(key);pickup(key)))
t.push((3,goto(A,B);pickup(key);goto(B,A)))

(3,goto(A,B);pickup(key);goto(B,A)) <- t.pop()
mark(3)
t.push((2,goto(A,B);pickup(key);goto(B,A);pickup(key)))
t.push((3,goto(A,B);pickup(key);goto(B,A);goto(A,B)))
t.push((4,goto(A,B);pickup(key);goto(B,A);goto(A,C)))

(4,goto(A,B);pickup(key);goto(B,A);goto(A,C)) <- t.pop()
mark(4)
//4 is goal!
return goto(A,B);pickup(key);goto(B,A);goto(A,C)
```

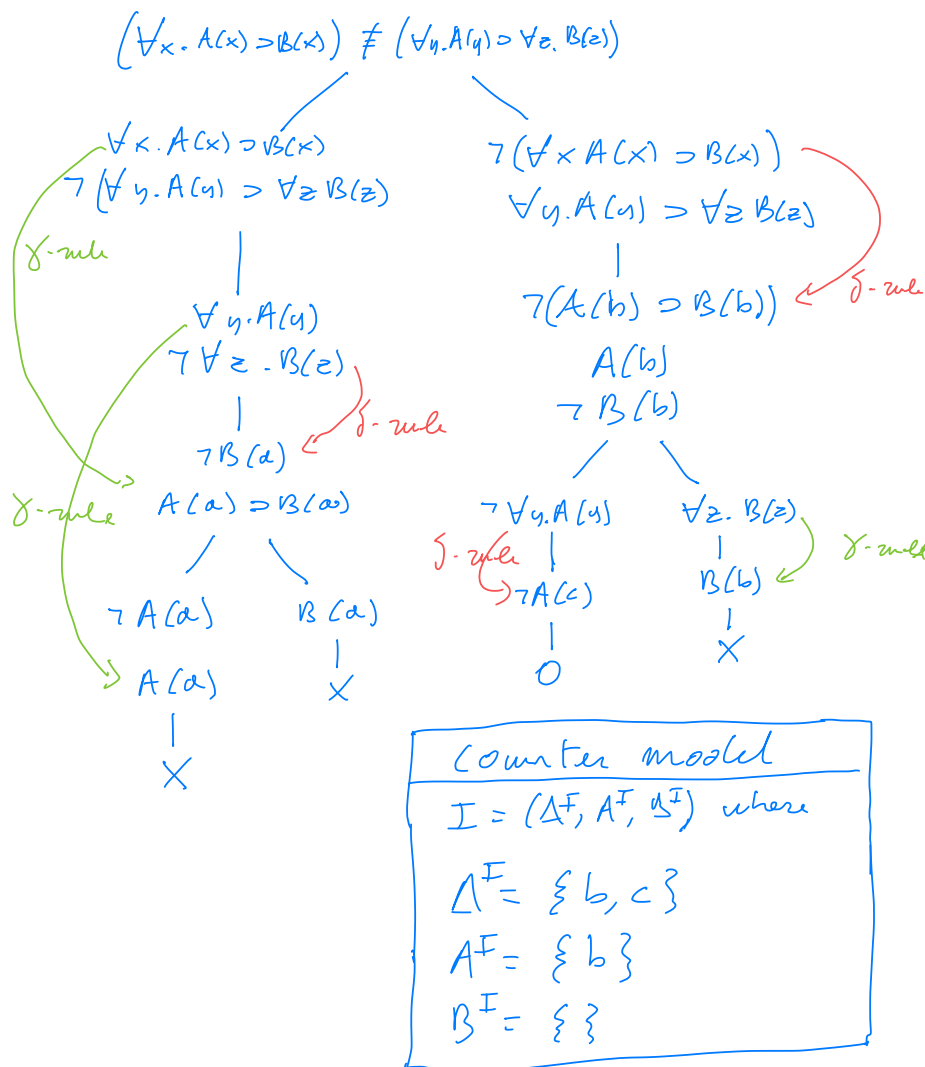
So we return the plan *goto(A,B);pickup(key);goto(B,A);goto(A,C)*, which indeed reaches the goal.

3 Exercise 3: solution

To show that $(\forall x.(A(x) \supset B(x))) \equiv ((\forall y.A(y)) \supset (\forall z.B(z)))$ is valid we try to show that

$$(\forall x.(A(x) \supset B(x))) \not\equiv ((\forall y.A(y)) \supset (\forall z.B(z)))$$

is unsatisfiable by using tableaux.



The tableau remains open, and a counter example can be build: $I = (\Delta^I, A^I, B^I)$ with $\Delta^I = \{c, b\}$, $A^I = \{c\}$, $B^I = \{\}$. I is indeed is a counter model: $I \not\models \forall x.(A(x) \supset B(x))$, since $A^I \not\subseteq B^I$, while $I \models (\forall y.A(y)) \supset (\forall z.B(z))$ since $\forall y.A(y)$ is false and hence the implication itself is trivially true.

4 Exercise 4: solution

We remind the DPLL algorithm based on unit propagation and splitting rule:

```
DPLL(phi, I')
  (psi, I) := UnitPropagation(phi, I');
  if psi contains {} then
    return ({{}}, emptyset)
  elseif psi = {} then
    return ( {}, I )
  else
    select a literal lambda in a clause C in psi; //splitting rule
    if DPLL(psi union {{lambda}}, I) = ( {}, I'' ) then return ( {}, I'' )
    else return DPLL(psi union {{not lambda}}, I)

UnitPropagation(phi, I)
  while phi contains a unit clause {lambda}
    phi := phi|lambda;
    if lambda = p, then I(p) := true;
    if lambda = not p, then I(p) := false
  end;
  return (phi, I)
```

We apply the DPLL algorithm to our case:

```
DPLL({ {A, B, ¬C}, {¬A, B, C}, {¬A}, {¬B, C} }, { }) →
UnitPropagation({ {A, B, ¬C}, {¬A, B, C}, {¬A}, {¬B, C} }, { }) → ({ {B, ¬C}, {¬B, C} }, {¬A})
DPLL({ {B, ¬C}, {¬B, C} }, {¬A}) →
  apply splitting rule, e.g., on B
UnitPropagation({ {B, ¬C}, {¬B, C}, {B} }, {¬A, B}) → ({ {C} }, {¬A, B})
DPLL({ {C} }, {¬A, B}) →
UnitPropagation(({ {C} }, {¬A, B}) → ( {}, {¬A, B, C} )
return ( {}, {¬A, B, C} )
```

Hence the formula is satisfiable, and a model for it is $A = false, B = true, C = true$.